

Crypto Order Book Engine

System Design and Architecture

A production-grade real-time order book built in Rust. WebSocket ingestion from Binance, async Tokio throughout, REST API with Axum. Every decision documented. Every tradeoff named.

Language: Rust | Runtime: Tokio | Exchange: Binance | API: Axum

Part 1 - What We Are Actually Building

Binance is a crypto exchange. When people buy and sell Bitcoin, those orders sit in a list called an order book. Bids on one side are people wanting to buy and what they are willing to pay. Asks on the other side are people wanting to sell and what price they want. That list changes thousands of times per second.

We are building a program that connects to Binance, receives those changes in real time, keeps its own copy of the order book in memory always up to date, and lets other programs ask what the order book looks like right now over HTTP.

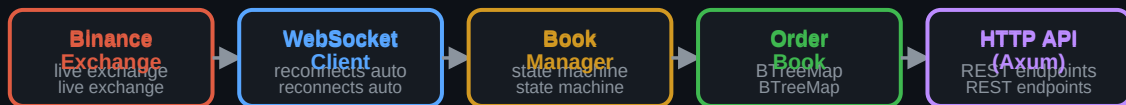


Figure 1 - Top-level data flow. Each box is an independent piece. Arrows show how data moves.

Part 2 - Cargo.toml: Every Dependency and Why

Every dependency is a bet. You are betting this tool is mature, maintained, and will not fight with the rest of the system. Here is every bet we made.

tokio

Rust by default does one thing at a time. If you are waiting for Binance to send a message, your program just sits there doing nothing. Tokio lets your program do many things at the same time: listen to Binance, handle an incoming HTTP request, run a timer, all on the same thread without blocking. Everything else depends on this being here.

tokio-tungstenite

Binance sends us data over a WebSocket, which is like a phone call you open once and both sides can talk back and forth as long as they want. This library speaks that protocol and is built to work with Tokio natively, so they share the same plumbing underneath.

serde and serde_json

Binance sends JSON text. Serde reads that text and fills in our Rust structs automatically. You describe the shape you expect and it handles the rest. The standard for JSON in Rust.

rust_decimal

Binance sends prices as strings like "29431.50000000". Convert that to a regular float and you get silent rounding errors. 0.1 plus 0.2 in floating point equals 0.30000000000000004, not 0.3. In financial software that error compounds with every operation. rust_decimal stores the number exactly as written. We never use floats for prices.

f64 float (WRONG)		rust_decimal (CORRECT)	
Binance sends:	"29431.50000000"	Binance sends:	"29431.50000000"
Stored as float:	29431.49999999996	Stored exactly:	29431.50000000
0.1 + 0.2 =	0.30000000000000004	0.1 + 0.2 =	0.3
After 1000 ops:	errors compound	After 1000 ops:	still exact

Floats store most decimals approximately. Financial software demands exact values.

Figure 2 - Float math is approximate. Decimal math is exact. This is not optional in finance.

thiserror

When something goes wrong, the response depends on what went wrong. Network dropped? Reconnect. Bad JSON? Skip that frame. Missed a sequence number? Throw everything away and rebuild. thiserror lets us name each failure type so the code that catches it knows exactly what to do.

axum

The HTTP server that handles incoming requests for book data. Built by the same team as Tokio so they work together natively without fighting.

tracing and tracing-subscriber

Our program does many things simultaneously. A regular print statement has no way to tell you which task it came from. tracing attaches context to every log message: which connection, which task, what it was doing. Essential when debugging a system under load.

reqwest

HTTP client for fetching the order book snapshot from Binance's REST API. Built on the same network stack as Axum so they share connection infrastructure.

Decision	We Chose	Alternative	Why
Price type	rust_decimal	f64 float	Floats round silently. Financial systems cannot have invisible roundings.
HTTP framework	axum	actix-web	actix-web uses its own actor runtime that conflicts with Tokio. axum uses Tokio.
Error handling	thiserror	anyhow	anyhow erases error types. We need to match on variants to choose a response.
Logging	tracing	println	println has no concept of async task context. tracing tracks which task is logging.

Part 3 - error.rs: The Failure Taxonomy

We defined all error types before writing any logic. If you define what can go wrong first, every piece of code knows exactly what failures it is allowed to produce. If you do it later, you get a scattered mess of inconsistent errors with no clear recovery story.

WebSocket Network dropped Reconnect	InvalidUrl Bad config at startup Crash immediately
Parse Unreadable JSON frame Skip, keep going	InvalidField Non-numeric price string Skip frame
SequenceGap Missed an update Rebuild book	Io OS network error Reconnect
SyncFailed Bad first-connect handshake Redo handshake	

Each variant name tells you what happened. Each recovery action is different.

Figure 3 - Each error type maps to a specific recovery action. They are not interchangeable.

WebSocket

The connection to Binance dropped. This is a network problem, not a data problem. Recovery: close the connection and open a new one. The book state is still valid as long as we can pick up from where we left off.

Parse

Binance sent us a JSON message we could not understand. One bad frame does not mean the stream is broken. Recovery: log the message, skip it, keep going.

SequenceGap

We received update number 100 and then update number 102, skipping 101. That missing update could be anything. We cannot reconstruct what changed. Recovery: throw away the entire book and rebuild it from a fresh snapshot.

SyncFailed

When we first connect there is a handshake that aligns our book with Binance. If that handshake fails the numbers do not add up. Recovery: start the handshake over.

InvalidUrl

The address we are trying to connect to is malformed. This only happens at startup with a bad configuration. Recovery: crash immediately with a clear message. There is no recovering from bad config, a human needs to fix it.

Part 4 - ws/mod.rs: The Channel Contract

The WebSocket client and the order book manager are two separate tasks that never call each other directly. They communicate through a queue. WsEvent defines everything that can ever go in that queue. It is the contract between them.

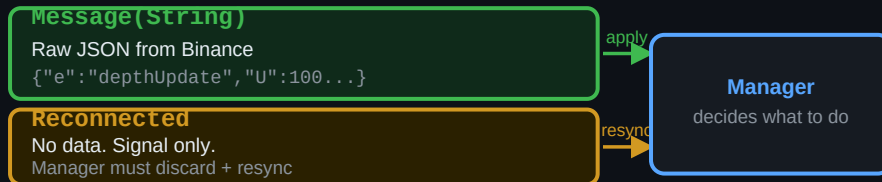


Figure 4 - Two variants, two completely different responses from the manager.

Message(String)

Binance sent us something. Here is the raw text exactly as received. We do not parse it here because the WebSocket client's only job is transport. If Binance changes their JSON format, only one file needs updating.

Reconnected

We lost the connection and re-established it. Everything the manager knew before is potentially from a dead stream. This signal goes into the queue before any new messages so the manager sees it first and resets before processing anything from the new connection.

Part 5 - ws/client.rs: The Reconnecting WebSocket Client

This piece talks to Binance. It runs forever in the background maintaining the connection and feeding messages into the queue. The entire rest of the system never touches a socket.



Figure 5 - Each failure doubles the wait time. Jitter staggers simultaneous reconnects.

Why it runs in its own task

When we call spawn, we hand the connection off to a background process and walk away. It runs forever feeding messages. If we did not do this, listening to Binance would block everything else in the program.

Why the wait time grows after each failure

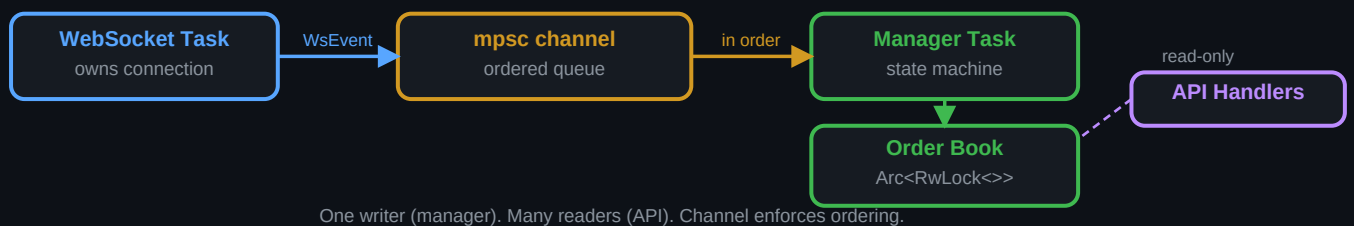
Imagine Binance's servers go down. If every client immediately retries every millisecond, they collectively flood the servers when they come back up, which might knock them down again. We wait, and we wait longer each time. After 30 seconds we stop growing the wait time, because waiting hours between retries is worse than occasionally being a little aggressive.

Why we add randomness to the wait

Without randomness, all clients that started together are on the same schedule: they all wait 1 second together, then all try together, then all fail together. Adding a small random offset spreads them out.

Why Reconnected must be first in the queue

When we reconnect, we send the Reconnected signal before any messages from the new connection. If we sent messages first, the manager would try to apply them to a book that was synced on a different session. The queue guarantees ordering, which is the guarantee we rely on.



One writer (manager). Many readers (API). Channel enforces ordering.

Figure 6 - One writer, one reader, one queue. The RwLock on the book allows many API readers at once.

Part 6 - ws/binance.rs: Parsing What Binance Actually Sends

This is the only file that knows about Binance's JSON format. Everything outside this file works with our own types and never sees field names like 'b', 'u', or 'U'.

```
{
  "e": "depthUpdate",
  "E": 1699000000000,
  "s": "BTCUSDT",
  "U": 100,
  "u": 103,
  "b": [ ["29500.00", "1.245"],
         ["29499.50", "0.000"] ],
  "a": [ ["29501.00", "0.310"] ]
}
```

-- event type
-- timestamp (milliseconds)
-- trading symbol
-- first sequence ID
-- last sequence ID
-- bids to update
-- qty=0 means DELETE
-- asks to update

Figure 7 - Every field annotated. We only use the ones relevant to book maintenance.

Why prices are strings in the JSON

Binance sends prices as text like "29500.00000000" rather than numbers. This is intentional: it lets them control the exact precision without relying on whatever floating point format the client uses. We parse them directly into `rust_decimal` which preserves that precision.

How Binance signals a deletion

When a price level needs to be removed from the book, Binance sends the same update format but with a quantity of zero. That zero is the signal. Our `is_removal()` method checks for it. If we missed this we would fill the book with ghost entries that do not exist on the exchange.

BEFORE update		AFTER update	
29,501	0.842	29,501	0.842
29,500	1.000	29,500	1.245
29,499	0.000	29,499	REMOVED
29,498	0.500	29,498	0.500

Update received: ["29499", "0.000"]
qty=0 = DELETE

Binance signals deletion with quantity zero, not a separate message.

Figure 8 - Before and after a zero-quantity update. The price level at 29,499 is deleted.

The two-layer design

We define two separate sets of types. `RawDepthUpdate` matches Binance's JSON exactly and is private to this file. `DepthUpdate` is our type with proper `Decimal` fields and clear names. The conversion between them is one function. If Binance renames a field, we change one line.

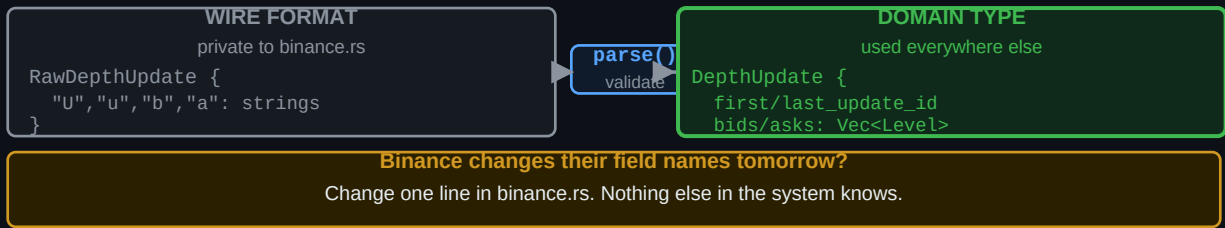


Figure 9 - Wire format stays private. Domain type is what the rest of the system uses.

Sequence numbers

Every update has a first sequence ID (U) and a last sequence ID (u). These are like page numbers in a book. The manager uses them to detect whether any updates were missed. This file's job is to carry them faithfully.

No gaps: each U = previous u + 1



Gap detected: expected U=108, got U=109

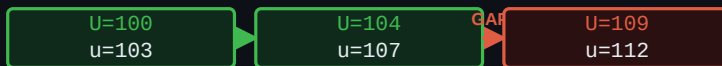


Figure 10 - Continuous sequence vs gap. One missing number means the book must be rebuilt.

Part 7 - orderbook/book.rs: The In-Memory Order Book

This is the heart of the system. Everything else exists to feed data into this structure or to read from it. It holds the current state of the market: every price where someone is willing to buy or sell and how much.

BIDS (buyers, highest first)			ASKS (sellers, lowest first)		
29,501.00	BEST BID	0.842	29,502.00	BEST ASK	0.310
29,500.00		1.245	29,503.00		2.100
29,499.50		3.100	29,504.50		0.750
29,498.00		0.500	29,507.00		5.000
29,495.00		12.30	29,510.00		8.200

Highest buyer at \$29,501. Lowest seller at \$29,502. Gap between them = spread.

Figure 11 - A live order book. Bids on the left sorted highest first. Asks on the right sorted lowest first.

Why BTreeMap and not HashMap

We need to do two things constantly: update a specific price level by its exact price, and find the best bid and best ask instantly. A HashMap gives fast lookup but stores entries in random order, so finding the best bid requires scanning everything. A BTreeMap stays sorted automatically with every insert and remove, so the best bid is always the last entry and the best ask is always the first.

HashMap (unordered)			BTreeMap (sorted)		
29498		0.500	29495		12.30
29501		0.842	29498		0.500
29495		12.30	29499		3.100
29500		1.245	29500		1.245
29499		3.100	29501	BEST BID	0.842
To find best: scan ALL entries			Best bid = last entry, always		

Figure 12 - HashMap forces a full scan to find best price. BTreeMap puts it at the end, always.

What the book looks like in memory

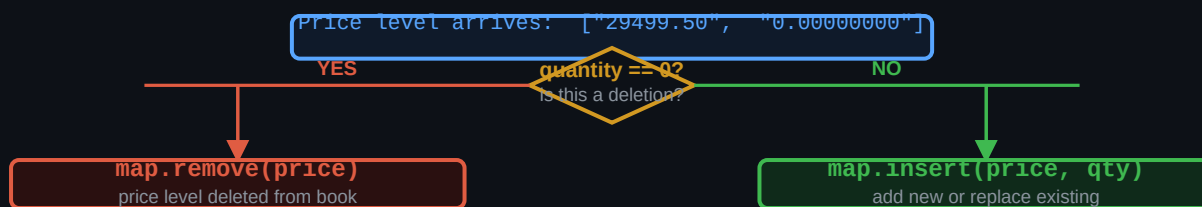
The bids side is one BTreeMap sorted ascending by price. The best bid is the last entry. We find it by calling `next_back()` which is one operation regardless of how many entries are in the map. The asks side is identical. Best ask is the first entry, found with `next()`.

	bids BTreeMap sorted low to high		asks BTreeMap sorted low to high	
	29,495	12.30	29,502	0.310
	29,498	0.500	29,503	2.100
	29,499	3.100	29,504	0.750
	29,500	1.245	29,507	5.000
ext_back() -> BEST BID	29,501	0.842	29,510	8.200
				next() -> BEST ASK

Figure 13 - Both maps sorted low to high. One call finds best bid (back) or best ask (front).

What happens when an update arrives

For each price level in the update we do exactly one of two things. If quantity is greater than zero we insert or overwrite that price. If quantity is zero we remove that price. That is the entire logic.

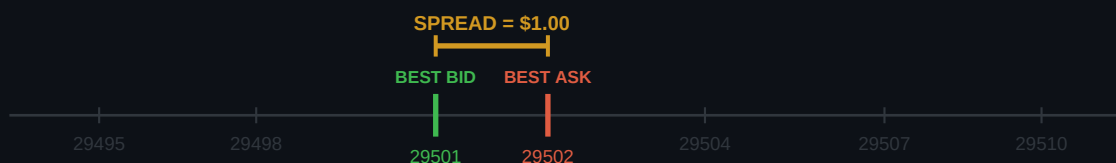


Two paths only. The BTreeMap handles new vs existing automatically.

Figure 14 - Two paths only. The BTreeMap handles new vs existing entries automatically.

The spread

The spread is the gap between the lowest seller and the highest buyer. It represents the cost of trading immediately: if you buy at the best ask and sell at the best bid right after, you lose exactly the spread. A tight spread means a healthy liquid market.

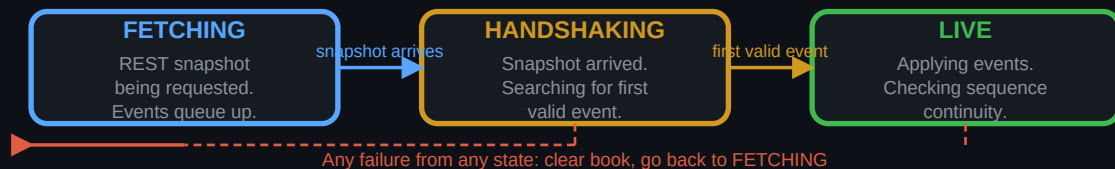


The spread is the cost of buying then immediately selling. Tight spread = liquid market.

Figure 15 - Spread shown as the gap on a price axis. Tighter spread = more liquid market.

Part 8 - orderbook/manager.rs: The Sync State Machine

The manager is the most important piece. It sits between the WebSocket client and the order book and answers one question at all times: is it safe to apply this update or not? Getting this wrong means the book silently diverges from reality and nobody knows until trades go wrong.



Reconnect signal, sequence gap, or failed handshake all lead to the same recovery.

Figure 16 - Three states. Any failure from any state resets to Fetching.

The three states

Fetching: we are asking Binance's REST API for a full snapshot. Events from the WebSocket are arriving and queuing up. We do not touch the book.

Handshaking: the snapshot arrived. We scan queued events to find the first one that picks up exactly where the snapshot left off.

Live: we found our starting point. Every update gets sequence-checked then applied.

Why the snapshot and stream overlap in time

You cannot pause Binance's WebSocket while you fetch the snapshot. It keeps sending no matter what. By the time the snapshot arrives, some events in the queue describe changes already in the snapshot. Others happened after. The handshake sorts them out using sequence numbers.

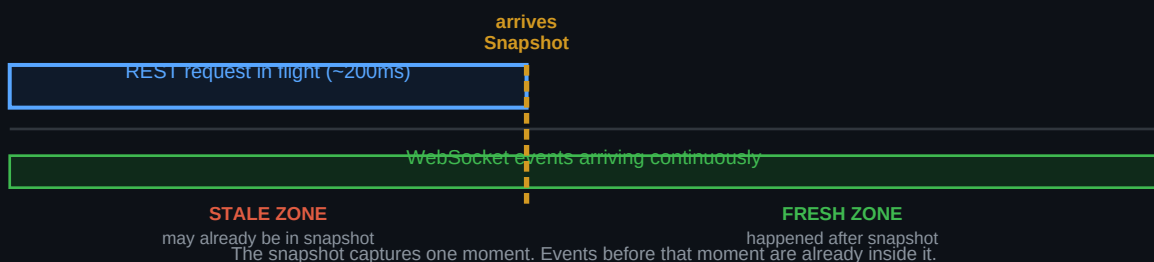


Figure 17 - REST request takes ~200ms. WebSocket events keep arriving the entire time.

Snapshot lastUpdateId = 1000		
U=980	u=995	STALE
U=996	u=1000	STALE
U=1001	u=1004	FIRST OK
U=1005	u=1008	PENDING
U=1009	u=1012	PENDING

Stale events are discarded. First event where $U \leq \text{lastId} + 1$ AND $u > \text{lastId}$ is the starting point.

Figure 18 - The queue when snapshot arrives. Stale events are discarded. The first valid one is the starting point.

The exact handshake rule

Each event has a first ID (U) and a last ID (u). The snapshot has lastUpdateId. Discard any event where u is less than or equal to lastUpdateId: those changes are already in the snapshot. Reject and resync if U is greater than lastUpdateId plus one: there is a gap before the first event. Apply the event if it passes both checks.

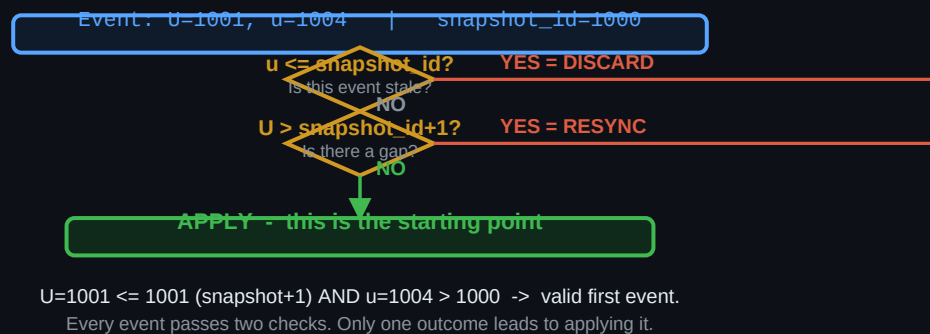


Figure 19 - Every event during handshake passes two checks. Only one outcome leads to applying it.

Live mode sequence checking

Once live, the check is simpler. Every event's first ID must equal the previous event's last ID plus one. If the numbers do not connect, we missed something. Clear the book and start over.

GOOD: each U = previous u + 1



BAD: expected U=108, got U=109 - one update was lost

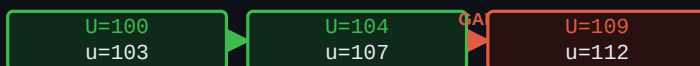
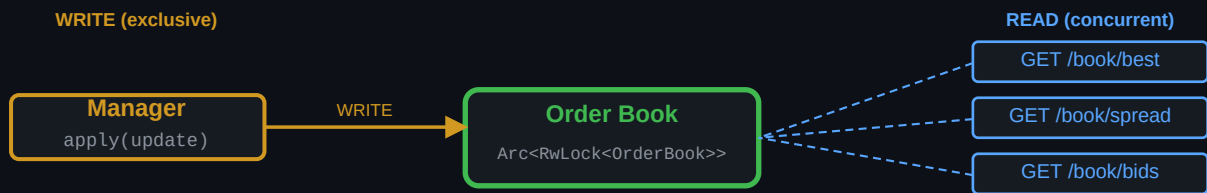


Figure 20 - Good sequence vs gap. One broken link in the chain means the whole book is wrong.

Why std::sync::RwLock instead of tokio's version

Applying an update to the book is pure computation. No network calls. No waiting for anything. It takes microseconds. We use the standard library RwLock because it is simpler and faster for pure computation. The rule we never break: take the lock, do the work, release it. Never wait for the network while holding it.



Many readers can run at once. The writer waits for all readers to finish.
`std::sync::RwLock` is correct here because book writes are pure computation - no awaiting.

Figure 21 - Many readers run simultaneously. The writer waits for readers to finish then writes alone.

Decision	We Chose	Alternative	Why
Lock type	<code>std::sync::RwLock</code>	<code>tokio::sync::RwLock</code>	Book operations are pure computation. std is simpler and faster when it comes to lock contention.
Snapshot buffering	mpsc internal queue	explicit Vec buffer	While we await the REST call, Tokio suspends our task. The channel buffer is a good fit for this use case.
Reconnect recovery	restart the <code>run()</code> loop	separate signal task	One loop restart keeps all state transitions in one place and eliminates the need for a separate task.

Part 9 - api/routes.rs: The HTTP API

The API is how the outside world reads the order book. Every endpoint takes a read lock on the book, reads what it needs, releases the lock immediately, then serializes to JSON. The lock is held for microseconds. Many requests can run at the same time.

GET	/health	Is the book synced? Returns 503 until first snapshot applied.
GET	/book/best	Best bid price, best ask price, spread, and last update ID.
GET	/book/snapshot	Top N levels on both sides (default 20, max 100).
GET	/book/bids	Top N bid levels only. Pass ?depth=50 for more.
GET	/book/asks	Top N ask levels only. Pass ?depth=50 for more.
GET	/book/spread	The current spread as a single decimal number.

All endpoints return JSON. No authentication - this is an internal service.

Figure 22 - Six endpoints. All GET, all read-only, all return JSON.

The health endpoint

GET /health returns 200 OK when the book has been synced at least once, and 503 Service Unavailable while we are still waiting for the first snapshot. Load balancers and monitoring systems use this to decide whether the instance is ready to receive traffic. A 503 means: do not send requests yet.

Depth limits

The snapshot and bids/asks endpoints accept a depth query parameter. The default is 20 levels. The maximum is capped at 100. Without the cap, a caller could request the entire book, which could have thousands of levels, and cause a slow response that blocks a read lock far longer than the microseconds we want.

The AppState pattern in Axum

Axum passes shared state to every handler by cloning it on each request. Our AppState holds an Arc, which is a reference-counted pointer. Cloning an Arc does not copy the book, it just increments a counter. So every handler gets its own handle to the same book cheaply.



The handler acquires a read lock, reads the book, releases the lock, serializes to JSON. Lock is held for microseconds. Many requests can run simultaneously.

Figure 23 - A request travels through Axum, hits the handler, acquires a read lock, reads the book, releases, returns JSON.

Decision	We Chose	Alternative	Why
----------	----------	-------------	-----

Response format	JSON via axum::Json	Custom binary format	JSON is readable and debuggable. Binary is faster but adds complexity
Depth cap	min(requested, 100)	No cap	Without a cap a single request can lock the book for milliseconds in the worst case
State sharing	Arc<RwLock<>> clone	Global singleton	Arc makes ownership explicit and testable. Globals are invisible dependencies

Part 10 - config.rs: Runtime Configuration

Configuration comes from environment variables. This is the standard approach for containerized services because it lets you change behavior without rebuilding the binary and without committing credentials to source control.



Environment variables are the standard way to configure containerized services.

Figure 24 - Environment variables flow into a typed Config struct at startup.

Why environment variables and not a config file

Config files get committed to source control, which means secrets end up in git history. They also require the file to be present on the machine, which means deployment has to place it correctly. Environment variables are set by the deployment system and never touch disk. In Docker you set them with `-e` or in `docker-compose.yml`.

Sensible defaults

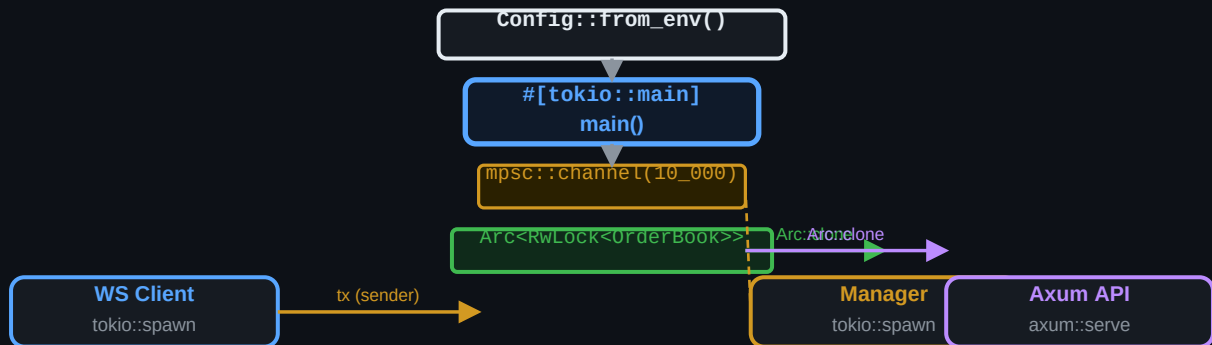
Every setting has a default that makes the system work out of the box. You can run it with zero configuration and it tracks BTCUSDT on port 3000. You only set environment variables when you want to change something.

The channel buffer size

`CHANNEL_BUFFER` controls how many WebSocket events can queue up while the manager is busy fetching a REST snapshot. The default is 10,000. At Binance's update rate of roughly 10 to 100 events per second, that is enough for a snapshot fetch that takes up to 1,000 seconds, far more than any reasonable fetch. If this fills up the WebSocket client task slows down, which is the correct backpressure response.

Part 11 - main.rs: Wiring Everything Together

main.rs is short by design. Its only job is to create each component, connect them to each other, and start the Axum server. All the real logic lives in the other modules.



`main()` creates everything, wires it together, then serves HTTP until process ends.
The book is the only shared state. Everyone else communicates through it or the channel.

Figure 25 - `main.rs` creates the book, channel, and three tasks. Then serves HTTP.

The startup sequence

First we read configuration from environment variables. Then we create the order book wrapped in an `Arc` so it can be shared. Then we create the `mpsc` channel, spawn the WebSocket client with the sender end, spawn the manager with the receiver end and a clone of the book `Arc`, give the API layer another clone of the book `Arc`, and finally bind and serve the HTTP server. The server runs in the foreground. When it exits the process ends.

Why the HTTP server runs in the foreground

We call `tokio::spawn` for the WebSocket client and manager because those are background tasks. We do not spawn the HTTP server because we want the process to stay alive as long as the server is running. If we spawned it, `main()` would return immediately and kill everything.

Part 12 - Dockerfile: Shipping the System

The Dockerfile uses two stages. The first stage compiles the binary using the full Rust toolchain. The second stage takes only the compiled binary and puts it in a minimal image with no compiler, no shell, and no extra tools.

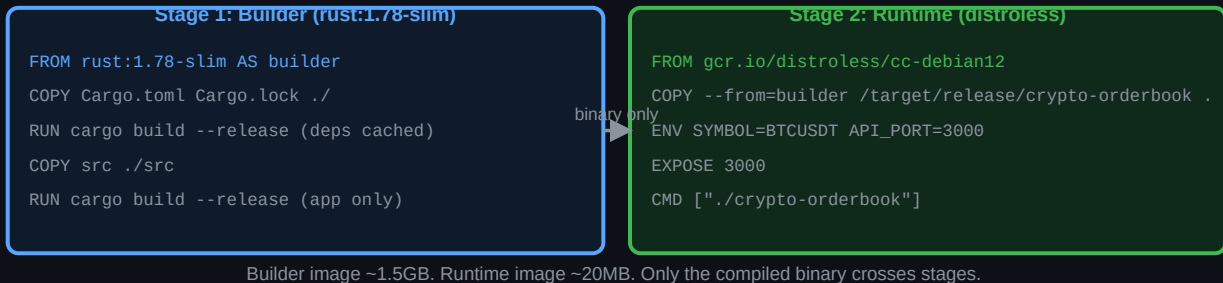


Figure 26 - Builder image is 1.5GB. Runtime image is ~20MB. Only the binary crosses stages.

Why two stages

The Rust compiler is enormous. If we used a single stage the production image would include the entire compiler, all build caches, and all source code. That is unnecessary weight and a larger attack surface. Multi-stage builds let us compile in one image and deploy from another.

Why we copy Cargo.toml first and compile deps separately

Docker builds layer by layer and caches each layer. If we copied all source files at once, any change to src/ would invalidate the dependency compilation layer and force all 200 dependencies to recompile from scratch. By copying Cargo.toml first and compiling a stub binary, the dependency compilation gets cached. Only the application code recompiles when src/ changes.

Why distroless as the runtime image

Distroless images contain only the minimum runtime libraries. No shell means no interactive access to a compromised container. No package manager means no way to install tools after the fact. The attack surface is as small as it can be.

Decision	We Chose	Alternative	Why
Build strategy	Multi-stage Docker	Single stage	Single stage ships the full compiler. Multi-stage ships only the binary.
Runtime image	gcr.io/distroless	ubuntu or alpine	Distroless has no shell and no package manager. Smaller and more secure.
Dep caching	Cargo.toml copied first	All at once	Copying all source at once invalidates the cache on every src change.

Part 13 - Production Roadmap: What a Real Trading System Needs

What we have built is a solid, correct foundation. A production trading system running at scale needs everything below. Each one is a significant engineering investment and each one solves a real problem that will surface when the system is under real load.

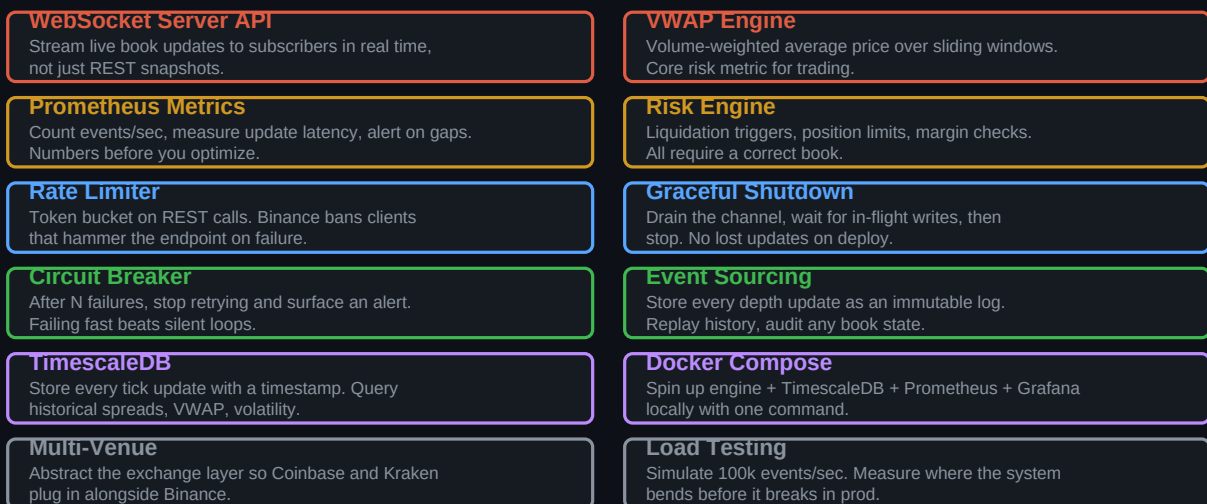


Figure 27 - Twelve production capabilities. Each solves a specific failure mode at scale.

WebSocket Server API

Right now clients poll our REST API for updates. In a real trading system every millisecond matters and polling is too slow. A server-side WebSocket lets clients subscribe to live book updates and receive them the moment they happen. Axum supports this with `axum::extract::ws`. The challenge is fan-out: one book update needs to go to all subscribers simultaneously, which requires a `tokio::sync::broadcast` channel and careful handling of slow subscribers that fall behind.

Prometheus Metrics

You cannot improve what you cannot measure. A production order book needs counters for events processed per second, gauges for current book depth and spread, histograms for how long each update takes to apply, and alerts when sequence gaps occur. The metrics crate plus prometheus exports give you this. The principle: measure first, then optimize. Never guess.

Rate Limiter on REST Calls

Binance bans clients that make too many REST requests in a short window. In our reconnect loop the snapshot fetch could fire rapidly during a crash loop and get the IP banned, making recovery impossible. A token bucket rate limiter allows a burst of requests then enforces a steady-state limit. The bucket refills at a fixed rate. Each request costs one token. If the bucket is empty you wait.

Circuit Breaker

A circuit breaker stops retrying after a certain number of consecutive failures and requires a manual reset or a timeout before trying again. Without one, a persistent failure like a misconfigured URL generates an infinite log-spamming retry loop that burns resources and obscures the root cause. The circuit breaker surfaces the problem clearly and stops the waste.

TimescaleDB for Tick Data

Everything in our system right now exists only in memory. If the process restarts, all history is gone. A time-series database like TimescaleDB, which is PostgreSQL with time-series compression built in, lets you store every depth update with a nanosecond timestamp. Then you can query: what was the spread one hour ago? What was the average bid depth yesterday? These queries power risk engines, backtesting, and audit trails.

VWAP Engine

Volume-weighted average price is the average price of all trades weighted by how much was traded at each price. It is the single most important metric for evaluating whether a trade executed at a good price. A VWAP engine runs over a sliding time window and updates continuously as new trades arrive. It requires a separate trade stream from Binance and a rolling accumulator that is efficient at the microsecond level.

Risk Engine

A risk engine sits above the order book and enforces position limits, margin requirements, and liquidation triggers. It reads the best bid and ask continuously and compares them against open positions. If a position would be underwater at the current price it triggers a liquidation. This is where the correctness of our order book becomes critical: a single wrong price level could trigger an incorrect liquidation or fail to trigger a necessary one.

Multi-Venue Support

Right now we speak only Binance's protocol. A real trading platform aggregates data from multiple exchanges to get the best available price across all venues. This requires abstracting the exchange layer behind a trait: each exchange implements `connect()`, `parse_update()`, and `fetch_snapshot()`. The manager, book, and API layer never change regardless of which exchange is underneath.

Graceful Shutdown

When the process receives `SIGTERM` from the operating system during a deployment, it should finish processing any events currently in flight before exiting. Without this, a deployment mid-update could leave the book in a partially applied state. Tokio provides `CancellationToken` for this. You broadcast a shutdown signal, tasks drain their queues, and the process exits cleanly.

Event Sourcing

Instead of storing only the current state of the book, store every single depth update as an immutable event with a timestamp. The current book state is then derived by replaying events from the beginning. This lets you reconstruct the exact state of the book at any point in time, audit exactly what happened

and when, and replay history to test new logic against real market data without going live.

Load Testing

Before putting this in front of real money, you need to know exactly where it breaks. A load test simulates 10,000 events per second hitting the manager while 1,000 concurrent API requests hit the HTTP layer. You measure where latency climbs, where the CPU saturates, and whether the RwLock becomes a bottleneck under concurrent read load. The tool for this in Rust is criterion for microbenchmarks and a custom harness with tokio for system-level load.

Decision	We Chose	Alternative	Why
WebSocket API	broadcast channel fan-out	polling	Polling adds latency. Broadcast delivers the update the instant it happens.
Metrics	Prometheus + Grafana	logs only	Logs tell you what happened. Metrics tell you what is happening right now.
Tick storage	TimescaleDB	PostgreSQL only	TimescaleDB compresses time-series data 10x better and makes it easier to query.
Multi-venue	exchange trait	hard-coded URLs	A trait means adding Coinbase is a new file, not a rewrite of the manager.